PROYECTO 3 · SPRINT 1

INTELLIGENT DOCUMENT ASSISTANT

DOCUMENTO DE ESTUDIO

Cómo funciona la biblioteca

La analogía del archivo: del PDF a una colección consultable, paso a paso.



Next.js 16 · TypeScript · PostgreSQL

El recorrido completo ya existe. Solo falta el bibliotecario.

LA IDEA CENTRAL

El archivo ya funciona; el bibliotecario todavía no interpreta

Intelligent Document Assistant es una aplicación para subir PDFs y consultarlos. En el Sprint 1 no hay búsqueda semántica ni respuestas generativas: hay un archivo que recibe un manuscrito, lo valida, lo desarma en páginas, lo guarda con su origen y lo muestra en una sala de lectura. Lo importante es que el recorrido completo ya existe.

> **La tesis del Sprint 1:** no necesitamos que el archivero entienda el manuscrito para probar que el archivo funciona. Necesitamos que el manuscrito llegue, se inspeccione, se desarme en folios numerados, se guarde en el armario y se pueda consultar. Cuando llegue la búsqueda semántica, cambiamos al bibliotecario sin mover los estantes.

El reparto de roles

EN EL ARCHIVO	EN LA APLICACIÓN	DÓNDE VIVE
Mensajero / formulario	La persona que elige y sube el PDF	`UploadForm.tsx`
Ventanilla de recepción	El Route Handler que recibe el archivo	`POST /api/documents`
Inspección del paquete	Validación de tamaño, tipo y cabecera	`lib/pdf/validation.ts`
Transcripción	Extracción de texto página por página	`lib/pdf/extract-text.ts`
Folios numerados	Cada página con su número y su texto	`DocumentPage`
Ficha del catálogo	Metadatos del documento y estado	`Document`
Armario	La base de datos PostgreSQL	`docker-compose.yml`
Sala de lectura	Biblioteca y vista de detalle	`page.tsx` , `DocumentDetail.tsx`
Sello «Procesado»	El estado del documento	`status = PROCESSED`
Índice de la sala	La lista de páginas del documento	`DocumentDetail.tsx`
Búsqueda semántica	Todavía no existe	Sprint 2 en adelante

EL VIAJE DE UN MANUSCRITO

Seis pasos desde que subís el PDF hasta que lo ves en la sala

Seguí el recorrido de un PDF cualquiera. En cada paso vas a ver qué pasa en el archivo y qué pasa exactamente en el código.

1

El mensajero entrega el manuscrito

En el archivo: el mensajero llega con un sobre y lo entrega en la ventanilla.

En la app: `UploadForm` acepta el PDF por clic o arrastre. La comprobación de tamaño y tipo en el navegador es solo para dar feedback rápido; la decisión real la toma el servidor. `DocumentLibrary.handleUpload` arma un `FormData` con el campo `file` y hace `POST /api/documents`.
El formulario usa `multipart/form-data`. El archivo viaja en memoria; el nombre original se conserva solo como metadato de visualización.

2

La ventanilla inspecciona el paquete

En el archivo: el archivero mira el paquete. ¿Hay un manuscrito? ¿Pesa demasiado? ¿La etiqueta dice manuscrito? ¿La portada tiene el sello `%PDF-` ?

En la app: `validatePdfUpload` comprueba que el campo sea un archivo, que no esté vacío, que no supere `MAX_PDF_SIZE_MB` (10 MB por defecto), que el MIME declarado sea `application/pdf` y que los primeros 1024 bytes contengan `%PDF-`. El límite de páginas se verifica después, cuando PDF.js ya leyó el documento.

```
if (value.size > limits.maxSizeBytes) throw new AppError("FILE_TOO_LARGE", ...);
if (declaredType !== "application/pdf") throw new AppError("INVALID_FILE_TYPE", ...);
if (!hasPdfSignature(bytes)) throw new AppError("INVALID_PDF_HEADER", ...);
```

3

El manuscrito se desarma en folios

En el archivo: el archivero saca la encuadernación y numera cada folio. El folio 1 sigue siendo el folio 1 cuando el manuscrito llegue a la sala.

En la app: `extractPdfPages` abre el documento con PDF.js, verifica `numPages` contra el límite de 50, recorre cada página con `getPage(pageNumber)` y `getTextContent()`, normaliza espacios y saltos de línea, y arma `{ pageNumber, text }`.

```
for (let pageNumber = 1; pageNumber <= pdf.numPages; pageNumber += 1) {
  const page = await pdf.getPage(pageNumber);
  const content = await page.getTextContent();
  pages.push({ pageNumber, text: normalizePageText(content.items) });
}
```

PDF.js corre en el servidor con el build `legacy`: la versión 6 usa `Promise.try`, que no existe en Node 22.13, y el bundle legacy incluye el polyfill. Ese detalle vive en `extract-text.ts` y `pdfjs-config.ts`.

EL VIAJE DE UN MANUSCRITO · CONTINUACIÓN

Del folio suelto a la sala de lectura

4

Si no hay letras, el archivo no inventa

En el archivo: si el sobre trae solo fotografías de las páginas, el archivero no puede transcribir letras que no están. No adivina ni inventa.

En la app: si ninguna página tiene texto, `extractPdfPages` lanza `NO_EXTRACTABLE_TEXT` y el endpoint responde **422** con el mensaje exacto: **«No se encontró texto seleccionable en el PDF.»**. No hay OCR en este sprint. Un PDF dañado recibe `CORRUPT_PDF`.

5

Los folios se guardan con su origen

En el archivo: cada folio va a una carpeta con el número del manuscrito y el número de página. La ficha del catálogo se escribe una sola vez, al final.

En la app: `createDocumentWithPages` abre una transacción con `prisma.$transaction`: crea el `Document` y luego inserta todas las `DocumentPage` con `createMany`. Si una página falla, el documento también se revierte. No quedan registros a medio guardar.

```
return prisma.$transaction(async (tx) => {
  const document = await tx.document.create({ data: { ...meta, status: "PROCESSED" } });
  await tx.documentPage.createMany({
    data: pages.map((page) => ({ documentId: document.id, ...page })),
  });
  return document;
});
```

6

La sala de lectura abre

En el archivo: la ficha aparece en el catálogo y cualquier persona puede pedir el manuscrito y leer sus folios.

En la app: el endpoint devuelve **201** con el resumen; la lista se actualiza y el aviso de éxito enlaza al detalle. `/documents/[id]` es un Server Component que consulta el documento y sus páginas ordenadas por `pageNumber`, y muestra un índice y una tarjeta por página.
La lista inicial se lee en el servidor; la isla cliente solo maneja la subida y el refresco. El navegador nunca recibe credenciales de la base.

El estado «Pendiente» no es una fila de la base. Mientras la request está en vuelo, la UI muestra un estado transitorio del cliente. En la base solo se persiste `PROCESSED`. El valor `FAILED` existe en el enum para un pipeline asíncrono futuro: la extracción actual ocurre antes de cualquier escritura, así que si falla, no hay documento que marcar.

EL MAPA DEL RECORRIDO

Quién hace qué en cada extremo

El mismo recorrido de los seis pasos, en una sola vista. La línea punteada marca la frontera entre el navegador y el servidor: ahí es donde el código deja de ser React y pasa a ser un Route Handler de Next.js.

NAVEGADOR

1. UploadForm

El usuario elige el PDF. La validación local de tamaño y tipo es solo UX; no es la autoridad.

↓

NAVEGADOR

2. DocumentLibrary → POST

Arma el `FormData` con el campo `file` y hace `POST /api/documents`. Viaja como `multipart/form-data`; el nombre original es solo un dato, nunca una ruta.

FRONTERA CLIENTE / SERVIDOR

SERVIDOR · ROUTE HANDLER

3. POST /api/documents

Orquesta la operación: lee el formulario, valida, extrae y persiste. Traduce cada error a una respuesta HTTP segura.

↓

SERVIDOR · VALIDACIÓN + EXTRACCIÓN

4. validatePdfUpload + extractPdfPages

Comprueba tamaño, MIME, cabecera y páginas. Después PDF.js lee cada página y conserva su número.

↓

SERVIDOR · PERSISTENCIA

5. createDocumentWithPages

Una transacción crea el `Document` y sus `DocumentPage`. PostgreSQL es el armario.

↓

NAVEGADOR · LECTURA

6. 201 + lista + detalle

La lista muestra la ficha; el detalle renderiza el índice y el texto de cada folio.

La clave de la arquitectura: el navegador no sabe cómo se extrae el texto. Solo conoce el contrato: manda un PDF y recibe metadatos. Por eso el Sprint 2 puede agregar chunking sin tocar el formulario de carga ni la sala de lectura.

LOS CONTROLES DE LA VENTANILLA

Qué se rechaza antes de tocar el armario

La interfaz valida para dar feedback rápido, pero el servidor repite todas las comprobaciones. El navegador no es confiable: el nombre, el MIME y el tamaño declarados pueden no coincidir con el contenido real.

10 MB
Tamaño máximo por defecto

50
Páginas máximas por defecto

%PDF-
Firma en los primeros 1024 bytes

application/pdf
MIME declarado que se acepta

CONTROL	EN EL ARCHIVO	CÓDIGO INTERNO	HTTP
Archivo faltante	No llegó ningún sobre.	`MISSING_FILE`	400
Archivo vacío	El sobre está vacío.	`EMPTY_FILE`	400
Demasiado grande	El paquete supera el peso máximo.	`FILE_TOO_LARGE`	413
Tipo incorrecto	La etiqueta no dice «manuscrito».	`INVALID_FILE_TYPE`	415
Cabecera inválida	La portada no tiene el sello del archivo.	`INVALID_PDF_HEADER`	400
Demasiadas páginas	El manuscrito tiene más folios de los permitidos.	`TOO_MANY_PAGES`	422
Sin texto	Solo hay imágenes; no hay letras que transcribir.	`NO_EXTRACTABLE_TEXT`	422
PDF dañado	El manuscrito está roto y no se puede abrir.	`CORRUPT_PDF`	422
Documento inexistente	Ese manuscrito no está en el catálogo.	`DOCUMENT_NOT_FOUND`	404
Error inesperado	Se cayó el archivo; no se muestra el detalle interno.	`INTERNAL_ERROR`	500

Los mensajes son para la persona, no para la terminal. El detalle técnico queda en el servidor; el cliente recibe un código y una frase comprensible. El caso más importante del sprint es el PDF sin texto: el archivo no inventa letras y responde con una explicación clara.

EL ARMARIO Y LA FICHA

Por qué cada folio conserva su página

La ficha del catálogo describe el manuscrito completo. Los folios guardan el texto real. La relación entre ambos es lo que permitirá citar una respuesta en el Sprint 4.

LA FICHA

`Document`

- `id`
- `originalFilename`
- `mimeType`
- `sizeBytes`
- `pageCount`
- `status` · PROCESSED / FAILED
- `errorMessage` opcional
- `createdAt`, `updatedAt`

1 — N

LOS FOLIOS

`DocumentPage`

- `id`
- `documentId`
- `pageNumber`
- `text`
- `unique(documentId, pageNumber)`
- `onDelete: Cascade`

> **La frase que importa:** si el PDF dice «El período de garantía del producto es de dos años», la respuesta del Sprint 4 debería poder citar `documento.pdf, página 1`. Si hubiéramos guardado todo el texto como un único bloque, sabríamos que la frase está en el PDF, pero no en qué página.

La transacción evita fichas huérfanas

Crear el `Document` y sus `DocumentPage` dentro de `prisma.$transaction` garantiza que no quede una ficha sin folios si falla una inserción. O se guarda todo, o no se guarda nada. La clave foránea con `onDelete: Cascade` completa la otra mitad: si se borra la ficha, sus folios desaparecen con ella. `@@unique([documentId, pageNumber])` impide dos folios con el mismo número.

> **El binario no se guarda.** El PDF se procesa en memoria y se descarta. Solo se conserva el nombre original como metadato saneado; se eliminan controles y separadores de ruta, y nunca se usa para construir una ruta del sistema de archivos.

LO QUE VIENE

Lo que el archivo todavía no hace

El Sprint 1 construye la infraestructura de ingesta y lectura. La inteligencia llega después, por partes, sin romper el contrato de la ventanilla.

OCR

No lee imágenes

Un PDF escaneado como imagen no tiene texto seleccionable. El archivo lo rechaza en vez de inventar una transcripción.

CHUNKING

No divide en fichas temáticas

El texto se guarda por página, no en fragmentos con tamaño y solapamiento. Eso llega en el Sprint 2.

EMBEDDINGS

No tiene índice semántico

No hay vectores ni búsqueda por similitud. PostgreSQL tiene la imagen compatible con pgvector, pero todavía no guarda vectores.

RAG + LLM

No responde preguntas

No hay OpenRouter, ni prompt con contexto, ni citas generadas. La sala de lectura muestra texto; no conversa.

SEGURIDAD

No hay credenciales ni salas privadas

No hay autenticación, permisos por documento ni aislamiento entre usuarios. Por eso esta versión es solo local.

PRODUCCIÓN

No hay límites de uso ni observabilidad

No hay rate limiting, métricas, colas ni despliegue. Es el foco del Sprint 7.

Lo que no cambia: el contrato de la ventanilla. El navegador seguirá mandando un PDF y recibiendo metadatos; el chunking es un detalle interno del servidor.

HOJA DE RUTA

Qué se agrega en cada sprint

La arquitectura está preparada para crecer por partes. Esto es lo que cambia y lo que queda igual.

SPRINT	SE AGREGA	EN EL ARCHIVO	QUEDA IGUAL
2	Chunking con tamaño y solapamiento	Fichas temáticas que conservan `documentId` y `pageNumber`	La ventanilla de carga y el contrato del endpoint
3	Embeddings y pgvector	Un índice por significado dentro de PostgreSQL	El armario, la ficha y los folios
4	RAG y OpenRouter	Un bibliotecario que recupera fragmentos y responde con citas	El origen por página y la sala de lectura
5	Evaluación, búsqueda híbrida y reranking	Un control de calidad de las respuestas	El pipeline de ingesta y el contrato
6	Auth, permisos e IDOR	Credenciales y salas privadas	El recorrido de carga y lectura
7	Producción y optimización	Límites, costos, observabilidad y despliegue	La arquitectura y la separación por capas

La regla de oro: el Sprint 2 no debería tocar `UploadForm` ni el contrato `POST /api/documents`. Si hay que cambiar la UI para agregar chunking, la frontera está mal puesta.

El próximo paso: el Sprint 2 convierte cada página en fichas temáticas con tamaño y solapamiento. Cada ficha conserva `documentId` y `pageNumber`, así que la cita futura no pierde el origen.

EL MAPA DE ARCHIVOS

Dónde vive cada parte del archivo

Si mañana tenés que explicar el proyecto, este es el orden en el que conviene abrir los archivos.

```
src/
├── app/
│   ├── api/documents/route.ts        ← la ventanilla
│   ├── api/documents/[id]/route.ts   ← la ficha en JSON
│   ├── documents/[id]/page.tsx       ← la sala de lectura
│   └── page.tsx                      ← la puerta
├── components/documents/
│   ├── UploadForm.tsx                ← el ingreso
│   ├── DocumentLibrary.tsx           ← la isla cliente
│   └── DocumentDetail.tsx            ← los folios
├── lib/
│   ├── pdf/validation.ts             ← la inspección
│   ├── pdf/extract-text.ts           ← la transcripción
│   ├── documents/create-document.ts  ← la transacción
│   └── documents/queries.ts          ← la lectura
└── generated/prisma/                 ← cliente generado

prisma/schema.prisma                  ← el armario
docker-compose.yml                    ← PostgreSQL + pgvector
samples/documento-de-prueba.pdf       ← manuscrito de prueba
```

Checklist del Sprint 1

- [ ] La aplicación y PostgreSQL levantan localmente con Docker.
- [ ] Se sube un PDF de texto y aparece en la biblioteca.
- [ ] La extracción conserva el texto y el número de página.
- [ ] Los errores de carga no dejan registros a medio guardar.
- [ ] Los archivos inválidos o fuera de los límites reciben un mensaje entendible.
- [ ] La interfaz se usa sin exponer claves ni acceso a la base en el navegador.

Cómo probarlo

Levantá el servidor y subí el PDF incluido en el repositorio. También podés regenerarlo con `npm run sample:pdf`.

```
npm run dev
# Abrir http://localhost:3000
# Subir samples/documento-de-prueba.pdf

# O probar la API directamente:
curl -F "file=@samples/documento-de-prueba.pdf;type=application/pdf" \
  http://localhost:3000/api/documents
```

PARA COMPROBAR QUE ENTENDISTE

Seis preguntas de repaso

Estas son las preguntas que conviene poder responder sin mirar el código. Si salen, entendiste el recorrido y la lógica del sprint.

1. ¿Qué diferencia práctica hay entre extraer texto de un PDF y hacer OCR?

Extraer texto lee las letras que el PDF ya contiene como texto. El OCR mira una imagen y trata de reconocer formas. En este sprint solo aceptamos PDFs con texto seleccionable; los escaneados se rechazan con un mensaje claro.

2. ¿Por qué guardamos el texto por página en vez de un bloque completo?

Porque la cita futura necesita documento + página. Si todo fuera un bloque, podríamos saber que la frase está en el PDF, pero no señalar dónde. `DocumentPage` conserva esa relación desde el primer paso.

3. ¿Por qué la validación debe repetirse en el servidor?

Porque el navegador no es confiable: el nombre, el MIME y el tamaño declarados pueden no coincidir con el contenido real. El servidor valida el archivo de verdad y aplica los límites antes de procesarlo.

4. ¿Qué problema evita guardar el documento y sus páginas en una transacción?

Evita fichas huérfanas o documentos sin folios si falla una inserción. O se guarda el documento con todas sus páginas, o no se guarda nada. No hay estados intermedios.

5. ¿Qué dato necesitamos conservar ahora para mostrar citas con página más adelante?

La relación entre cada fragmento y `documentId` + `pageNumber`. Hoy el texto ya se guarda con esa relación; en el Sprint 2 los chunks heredarán esos mismos metadatos.

6. Si un PDF se procesa correctamente pero sus páginas no contienen texto, ¿qué estado o mensaje debería ver el usuario?

No se persiste un documento: el servidor responde **422** con «No se encontró texto seleccionable en el PDF.». El estado `FAILED` está reservado para un pipeline asíncrono futuro; en este sprint el error se muestra antes de crear la ficha.

> **La respuesta corta:** si no hay texto, no hay ficha. El archivo no inventa.

Para revisar el código

1. `src/app/api/documents/route.ts` — orquestación y errores.
2. `src/lib/pdf/validation.ts` — validación real.
3. `src/lib/pdf/extract-text.ts` — recorrido página por página.
4. `src/lib/documents/create-document.ts` — transacción.
5. `prisma/schema.prisma` — modelo y restricciones.

GLOSARIO

Los términos que conviene tener a mano

PDF

Un formato de documento que puede contener texto seleccionable, imágenes o ambas cosas. No todo PDF tiene texto que se pueda extraer.

Texto seleccionable

Las letras que el PDF guarda como texto y que un extractor puede leer. Si el PDF es una foto de una página, no hay texto seleccionable.

OCR

Reconocimiento óptico de caracteres: mirar una imagen y adivinar las letras. Queda fuera del Sprint 1.

MIME

El tipo declarado del archivo, por ejemplo `application/pdf`. Es una pista, no una garantía: por eso también se revisa la cabecera.

Cabecera `%PDF-`

La firma que identifica un PDF. Se busca en los primeros 1024 bytes, porque el estándar permite bytes previos.

Extracción

Leer el texto de cada página y conservar su número. No es lo mismo que renderizar o interpretar el documento.

Chunk

Un fragmento de texto más pequeño que una página. Llega en el Sprint 2 para recuperar partes relevantes sin enviar documentos completos al modelo.

Embedding

Una representación numérica del significado de un texto. Permite buscar por similitud semántica en el Sprint 3.

pgvector

Una extensión de PostgreSQL para guardar vectores y buscar por distancia. La imagen de Docker ya es compatible, pero este sprint no la usa.

RAG

Retrieval-Augmented Generation: recuperar fragmentos relevantes y usarlos como contexto para generar una respuesta. Es el objetivo del Sprint 4.

Transacción

Un grupo de operaciones que se confirman o se revierten juntas. Evita fichas sin folios o folios sin ficha.

`PROCESSED` / `FAILED`

Los estados posibles de un documento. Hoy solo se persiste `PROCESSED`; `FAILED` queda para un pipeline asíncrono futuro.

> **La frase para llevarte:** el Sprint 1 no construyó un bibliotecario; construyó el archivo completo alrededor del bibliotecario. Cuando llegue la búsqueda semántica, el manuscrito ya sabe llegar, guardarse y encontrarse.